

Projektdokumentation

Entwicklung einer Software zur Lagerverwaltung von medizinischem Verbrauchsmaterial in einem Krankenhaus

JavaFX · MySQL · MVC- und DAO-Architektur

Verfasser	Sascha Schulz
Gruppe	2551
Ausbildungsberuf	Wirtschaftsinformatiker (WI)
Ausbildungsstätte	SRH Berufliche Rehabilitation GmbH
Betreuerin / Ausbilderin	Frau Cramer
Projektzeitraum	04.05.2026 – 26.06.2026 (6 Wochen, 180 h geplant)
Technologien	Java 17, JavaFX 21, MySQL 8, JDBC, MVC-Pattern
Abgabedatum	17.07.2026 (Moodle-Upload als PDF)

Inhaltsverzeichnis

1 Einleitung 5

- 1.1 Projektbeschreibung 5
- 1.2 Projektziel 5
- 1.3 Projektumfeld 5
- 1.4 Projektbegründung 6

2 Projektplanung 6

- 2.1 Projektphasen 6
- 2.2 Ressourcenplanung 6
- 2.3 Entwicklungsprozess 7

3 Analysephase 7

- 3.1 Ist-Analyse 7
- 3.2 Anwendungsfälle 7
- 3.3 Lastenheft 8

4 Entwurfsphase 8

- 4.1 Datenbankentwurf 8
- 4.2 Architekturentwurf (MVC + DAO) 9
- 4.3 Entwurf der Benutzeroberfläche 10
- 4.4 Pflichtenheft 11

5 Implementierungsphase 11

- 5.1 Datenbank und JDBC-Verbindung 11
- 5.2 DAO-Schicht und Model-Klassen 12
- 5.3 Benutzeroberfläche und CRUD 13
- 5.4 Such- und Filterfunktion 13
- 5.5 Mindestbestand-Warnung und Lagerbuchungen 14

6 Abnahme- und Testphase 14

- 6.1 Testfälle und Testergebnisse 14
- 6.2 Soll-Ist-Vergleich 15
- 6.3 Abnahme und Übergabe 15

7 Dokumentation 15

- 7.1 Benutzerhandbuch 15
- 7.2 Entwicklerdokumentation 16

8 Fazit 16

- 8.1 Soll-Ist-Vergleich der Projektziele 16
- 8.2 Lessons Learned 16
- 8.3 Ausblick 17

Abgabeerklärung 18

Quellenverzeichnis 19

Anhang 20

- Anhang A – ER-Diagramm 20
- Anhang B – SQL-Datenbankskript 20

Anhang C – Benutzerhandbuch	22
Anhang D – Wochenplan.....	24
Anhang E – Projektantrag.....	24
Anhang F – Glossar.....	24

Abbildungsverzeichnis

Abbildung 1: Use-Case-Diagramm der Hauptfunktionen	8
Abbildung 2: ER-Diagramm der Datenbank (6 Tabellen, 3NF)	9
Abbildung 3: Schichtenarchitektur (MVC mit DAO-Schicht)	10
Abbildung 4: Entwurf des Hauptfensters (Bestandsübersicht)	11
Abbildung 5: ER-Diagramm der Datenbank (Anhang A)	20

Tabellenverzeichnis

Tabelle 1: Beteiligte Rollen und Auftragsschnittstellen	5
Tabelle 2: Projektphasen und Stundenverteilung	6
Tabelle 3: Risiken und Gegenmaßnahmen	6
Tabelle 4: Pflichtanforderungen laut Ausbildungsrichtlinie	8
Tabelle 5: Krankenhausspezifische Zusatzanforderungen	8
Tabelle 6: Datenbanktabellen im Überblick	9
Tabelle 7: Zuordnung der Klassen zu den MVC-Schichten	13
Tabelle 8: Definierte Testfälle (Auszug)	14
Tabelle 9: Soll-Ist-Vergleich der Anforderungen	15

Quellcodeverzeichnis

Listing 1: CREATE-Statement der Tabelle materialien	11
Listing 2: Singleton-Datenbankverbindung (DBConnection)	12
Listing 3: CRUD-Beispiel: create() in MaterialDAO	12
Listing 4: Gemeinsame Schnittstelle GenericDAO<T>	13
Listing 5: Filterung der Bestandsübersicht nach Name und Station	13
Listing 6: Mindestbestand-Warnung über eine RowFactory	14
Listing 7: Datenbankschema – CREATE-Statements aller sechs Tabellen	21

1 Einleitung

1.1 Projektbeschreibung

In der DOIT-Projektphase der SRH Beruflichen Rehabilitation GmbH entstand eine Desktop-Anwendung für die Lagerverwaltung von medizinischem Verbrauchsmaterial im Krankenhaus – etwa Verbandsmaterial, Spritzen oder Einmalhandschuhe. Umgesetzt wurde sie mit Java 17 und JavaFX für die Oberfläche, die Daten liegen in einer MySQL-Datenbank. Die Anwendung läuft lokal als Einzelplatzlösung und kommt ohne Internetanbindung aus.

Das Personal sieht damit jederzeit, welche Bestände auf den einzelnen Stationen vorhanden sind. Wird ein Material knapp, weist die Anwendung darauf hin; jede Ein- und Ausbuchung wird protokolliert.

1.2 Projektziel

Ziel war eine Anwendung, die sich ohne Einarbeitung bedienen lässt. Das Krankenhauspersonal soll damit Materialien anlegen, ändern und löschen, Bestände einsehen, nach dem Materialnamen suchen und nach Station filtern können; sinkt ein Bestand unter den Mindestbestand, erscheint eine Warnung. Vorgegeben waren dazu eine MySQL-Datenbank mit mindestens fünf Tabellen, vollständiges CRUD, eine MVC-Architektur sowie eine Such- bzw. Filterfunktion – diese vier Pflichtenforderungen bilden den Kern des Projekts.

1.3 Projektumfeld

Das Projekt wurde als Einzelprojekt in der WI-Ausbildung (Gruppe 2551) durchgeführt. Betreut, bewertet und abgenommen wird es von Frau Cramer. Inhaltlich orientiert es sich an einem realistischen Fall aus dem Krankenhausalltag. Auftraggeber und Endnutzer – Krankenhausleitung sowie Pflege- und Stationspersonal – sind fiktiv und dienen nur der praxisnahen Darstellung.

Tabelle 1: Beteiligte Rollen und Auftragsschnittstellen

Rolle	Person	Funktion
Betreuerin / Ausbilderin	Frau Cramer	Fachliche Begleitung, Bewertung, Abnahme
Fiktiver Auftraggeber	Krankenhausleitung	Anforderungssteller für die Software
Fiktive Endnutzer	Pflege- und Stationspersonal	Tägliche Nutzung der Materialverwaltung

1.4 Projektbegründung

In vielen Häusern wird Verbrauchsmaterial bis heute von Hand in Listen oder Tabellen gepflegt. Das kostet Zeit und ist fehleranfällig. Wer wissen will, wie viel auf einer Station noch liegt, muss erst die Liste durchgehen; eine Warnung bei knappem Material gibt es nicht, und im Nachhinein lässt sich kaum nachvollziehen, wer was entnommen hat. Genau hier setzt die Anwendung an. Sie verringert Fehlbestände, macht Material schneller auffindbar und gibt der Verwaltung eine Historie, die sich jederzeit prüfen lässt.

2 Projektplanung

2.1 Projektphasen

Das Projekt gliedert sich in sieben Phasen mit einem geplanten Gesamtaufwand von 180 Stunden. Bei sechs Wochen Anwesenheit zu je 32 Stunden ergeben sich 192 Stunden; die übrigen 12 Stunden sind als Puffer eingeplant. Die Aufteilung folgt dem Projektantrag (v2) und dem SRH-Leitfaden.

Tabelle 2: Projektphasen und Stundenverteilung

#	Phase	Inhalt	h
1	Analyse & Planung	Anforderungen, ER-Diagramm, DB-Schema, Wochenplan	20
2	Datenbank	SQL-Skripte, 6 Tabellen, Testdaten, JDBC-Verbindung	20
3	DAO & Model	MVC-Architektur, alle DAO- und Model-Klassen	20
4	UI-Grundfunktionen	Hauptfenster, CRUD für alle 6 Entitäten, Tabellen, Dialoge	40
5	UI erweitert	Suche nach Name, Filter nach Station, Mindestbestand-Warnung	25
6	Test & Bugfixes	Funktionen testen, Fehler beheben, Code-Review	25
7	Dokumentation	Benutzerhandbuch, Projektdoku, Abgabe (Moodle)	30
Σ	Gesamt		180

2.2 Ressourcenplanung

Als Einzelprojekt lagen Analyse, Entwicklung, Test und Dokumentation in einer Hand. An Sachmitteln genügte ein privater Laptop (Windows 10/11, ≥ 8 GB RAM); dazu kamen Java 17, JavaFX 21, MySQL 8 mit Connector/J, IntelliJ IDEA, Maven und Git – allesamt kostenfrei. Zur Übung der Kalkulation wurde zusätzlich eine fiktive Kostenrechnung aufgestellt: 150 Stunden Projektarbeit (alle Phasen außer der Dokumentation) zu einem angenommenen Satz von 80,00 €/h ergeben mit anteiligen Sachkosten rund 12.150,00 €. In der Ausbildung fallen diese Kosten natürlich nicht an.

Tabelle 3: Risiken und Gegenmaßnahmen

Risiko	Wahrsch.	Gegenmaßnahme
JDBC-Verbindungsprobleme	Mittel	Früher Test, Singleton-DBConnection
JavaFX-Komplexität (TableView)	Hoch	Einfache Layouts zuerst, schrittweise erweitern
Zeitverzug in Phase 4	Mittel	12 h Puffer, Kernanforderungen zuerst
Datenverlust im Test	Niedrig	Testdatenbank trennen, Backups

2.3 Entwicklungsprozess

Entwickelt wurde schrittweise entlang der Phasen. Den Anfang machten das Datenmodell und die Datenzugriffsschicht, erst danach kam die Oberfläche dazu. Bei der Oberfläche wurden zunächst die Kernmodule Materialien, Kategorien und Stationslager fertiggestellt, die optionalen Module Lieferanten, Bestellungen und Bestandsbewegungen folgten später. Auf diese Weise stand früh ein lauffähiger Kern, an dem sich die Pflichtenforderungen überprüfen ließen. Wichtige Entscheidungen und Abweichungen wurden im Projekttagbuch festgehalten.

3 Analysephase

3.1 Ist-Analyse

Ausgangspunkt ist die manuelle Materialverwaltung per Liste oder Tabellenkalkulation. Wer den Bestand einer einzelnen Station – etwa Station 1, OP, ICU oder Zentrallager – braucht, muss ihn sich erst zusammensuchen. Eine automatische Warnung beim Unterschreiten des Mindestbestands fehlt ebenso wie eine lückenlose Buchungshistorie, mit der sich Entnahmen nachverfolgen lassen. Bei der Pflege schleichen sich leicht Fehler ein, und gezielt nach Station oder Materialart filtern lässt sich ohnehin nicht.

3.2 Anwendungsfälle

Aus der Ist-Analyse ergaben sich die zentralen Anwendungsfälle. Das Personal legt Materialien an, bearbeitet und löscht sie und sieht in der Bestandsübersicht den je Material und Lager berechneten Bestand. Es sucht nach dem Namen, filtert nach Station und erkennt an der Warnung, wenn ein Bestand knapp wird. Hinzu kommen das Erfassen von Bestellungen sowie das Buchen von Ein- und Ausgängen über die Bestandsbewegungen; eine Entnahme wird als Ausgang, eine Umlagerung zwischen zwei Lagern als Ausgang im einen und Eingang im anderen Lager gebucht.

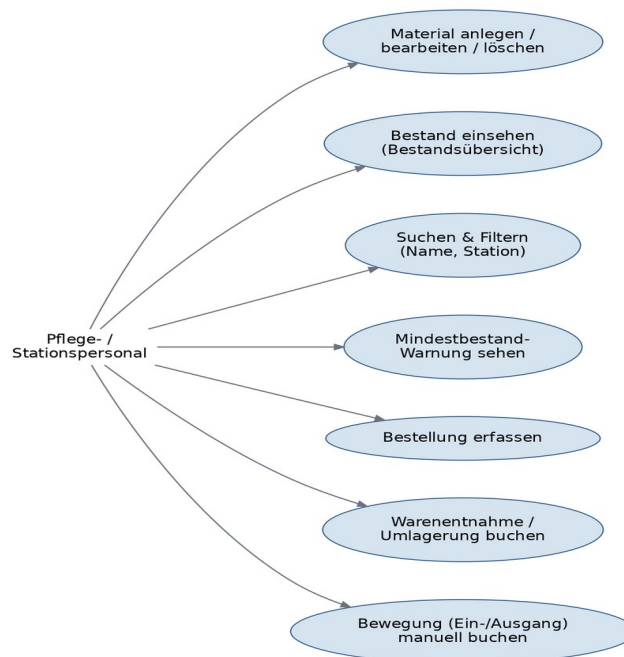


Abbildung 1: Use-Case-Diagramm der Hauptfunktionen

3.3 Lastenheft

Das Lastenheft fasst die Anforderungen zusammen. Die Pflichtanforderungen ergeben sich aus der Ausbildungsrichtlinie, die Zusatzanforderungen aus dem Krankenhauskontext.

Tabelle 4: Pflichtanforderungen laut Ausbildungsrichtlinie

Pflichtanforderung	Beschreibung
Datenbankanbindung	MySQL-Datenbank mit mindestens 5 Tabellen
CRUD-Operationen	Erstellen, Lesen, Ändern, Löschen für alle Entitäten
MVC-Architektur	Klare Trennung von Model, View und Controller
Such- und Filterfunktion	Suche nach Materialname, Filter nach Station

Tabelle 5: Krankenhausspezifische Zusatzanforderungen

Zusatzanforderung	Beschreibung
Mindestbestand-Warnung	Visuelle Hervorhebung bei Unterschreitung
Buchungshistorie	Protokoll aller Ein- und Ausgänge
Stationsfilter	Anzeige nach Krankenhaus-Station
Benutzerfreundlichkeit	Intuitive Oberfläche für nicht-technisches Personal

4 Entwurfsphase

4.1 Datenbankentwurf

Auf Basis der Anforderungen wurde das Datenmodell in sechs normalisierten Tabellen (3. Normalform) entworfen. Der aktuelle Bestand steht nicht als Feld im Material, sondern wird

aus den Bewegungen berechnet – Summe der Eingänge minus Summe der Ausgänge. Dadurch kann er nicht von der tatsächlichen Buchungslage abweichen, und die Historie bleibt vollständig nachvollziehbar.

Tabelle 6: Datenbanktabellen im Überblick

Tabelle	Zweck
kategorien	Materialgruppen (z. B. Verband, Injektion)
lieferanten	Medizinische Lieferanten mit Kontaktdaten
stationslager	Stationen und Lagerorte im Krankenhaus
materialien	Stammdaten aller Materialien inkl. Mindestbestand
bestellungen	Bestellhistorie mit Status und Lieferdatum
bestandsbewegungen	Buchungshistorie (EINGANG/AUSGANG); Bestand daraus berechnet

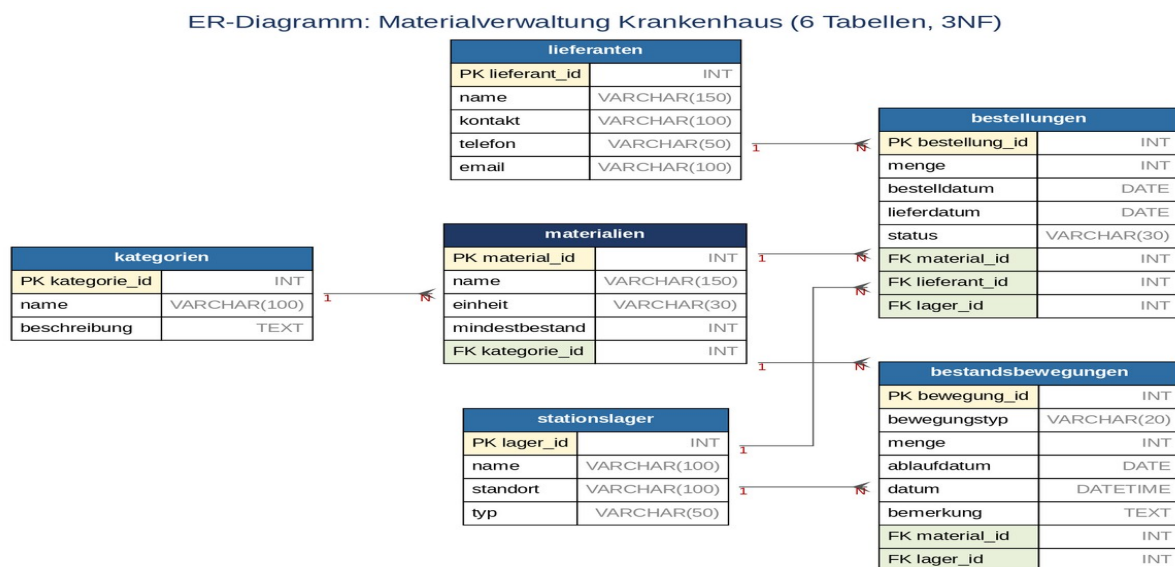


Abbildung 2: ER-Diagramm der Datenbank (6 Tabellen, 3NF)

4.2 Architekturentwurf (MVC + DAO)

Die Anwendung folgt dem Muster Model-View-Controller, ergänzt um eine DAO-Schicht für den Datenbankzugriff. Die View (JavaFX/FXML) zeigt Daten an und nimmt Eingaben entgegen, der Controller steuert die Bedienlogik, die DAO-Klassen kapseln sämtliche SQL-Zugriffe, und die Model-Klassen sind einfache Datenobjekte (POJOs). Weil jede Schicht eine klar umrissene Aufgabe hat, bleibt der Code überschaubar und lässt sich später leichter anpassen.

Alle sechs DAO-Klassen teilen denselben CRUD-Vertrag. Damit dieser nur an einer Stelle steht, wurde er in der Schnittstelle `GenericDAO<T>` zentral definiert. Eine Schnittstelle legt lediglich die Methodenköpfe fest, nicht deren Inhalt; jede Klasse, die sie implementiert, muss die Methoden selbst ausprogrammieren. Der Typ-Platzhalter `<T>` aus den Java-Generics wird

dabei erst beim Implementieren mit der jeweiligen Modellklasse belegt (etwa MaterialDAO implements GenericDAO<Material>). Eine einzige Schnittstelle reicht so für alle Entitäten.

Auch die Controller-Schicht wurde aufgeteilt. Der MainController ist nur noch Verteiler – er nimmt die Aktionen der Oberfläche entgegen und gibt sie an die zuständige Klasse weiter. Der immer gleiche Ablauf zum Anlegen, Bearbeiten, Löschen und Laden steckt in der abstrakten Basisklasse EntityCrud<T>; pro Reiter kommt eine kleine, gleich aufgebaute Klasse wie MaterialCrud hinzu, die nur die Besonderheiten ergänzt. So schrumpfte der MainController von rund 700 auf etwa 330 Zeilen, und wiederkehrender Code steht nur noch an einer Stelle.

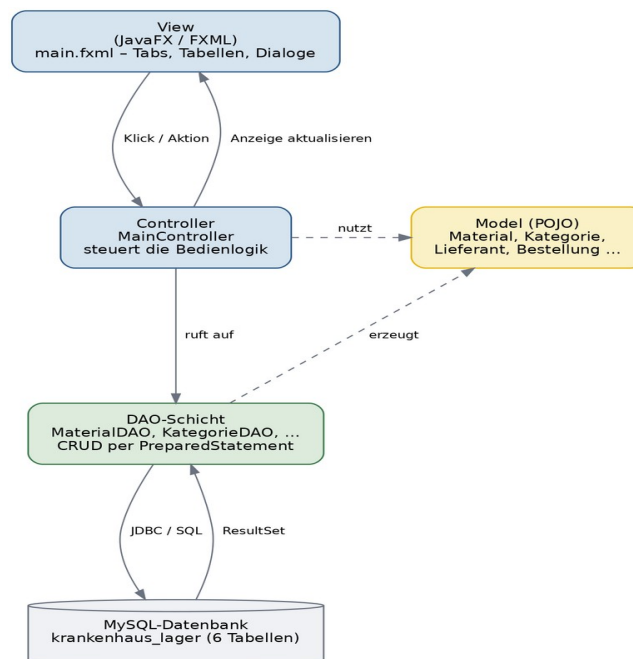


Abbildung 3: Schichtenarchitektur (MVC mit DAO-Schicht)

4.3 Entwurf der Benutzeroberfläche

Die Oberfläche besteht aus einem Hauptfenster mit sieben Registerkarten (TabPane). Auf der ersten Karte „Bestandsübersicht“ liegt oben eine Such- und Filterleiste mit einem Suchfeld für den Namen und einer Auswahl der Station, darunter die Bestandstabelle; Zeilen mit zu niedrigem Bestand werden rot hervorgehoben. Die übrigen Karten – Materialien, Kategorien, Stationslager, Lieferanten, Bestellungen und Bestandsbewegungen – bieten je Entität die Schaltflächen Anlegen, Bearbeiten und Löschen.

Materialverwaltung Krankenhaus - Bestandsuebersicht

Bestandsuebersicht
Materialien
Kategorien
Stationslager
Lieferanten
Bestellungen
Bestandsbewegungen

Suche: Materialname ...
Station v
Suchen
Zuruecksetzen

Material	Einheit	Bestand	Mindest	Status
Einmalhandschuhe M	Stk	420	100	OK
Mullkompressen	Pkg	18	50	WARNUNG
Einwegspritzen 5ml	Stk	250	80	OK
Desinfektionsmittel	Fl	12	30	WARNUNG

Aktualisieren
Warenentnahme
Umlagerung

Rot markierte Zeile = Bestand <= Mindestbestand (Status WARNUNG)

Abbildung 4: Entwurf des Hauptfensters (Bestandsübersicht)

4.4 Pflichtenheft

Das Pflichtenheft hält fest, wie die Anforderungen technisch umgesetzt werden. Die sechs Tabellen bilden das MySQL-Schema, die CRUD-Operationen laufen je Entität über PreparedStatements, und die Struktur folgt dem MVC-Muster mit DAO-Schicht. Die Bestandssicht wird nach Name und Station gefiltert, die Mindestbestand-Warnung übernimmt eine RowFactory, die betroffene Zeilen einfärbt. Die optionalen Module Lieferanten, Bestellungen und Bestandsbewegungen sind erst vorgesehen, sobald der Kern vollständig steht.

5 Implementierungsphase

Dieses Kapitel zeigt die Umsetzung an einigen repräsentativen Code-Auszügen; der vollständige Quellcode liegt dem Abgabepaket bei.

5.1 Datenbank und JDBC-Verbindung

Das CREATE-Skript legt alle sechs Tabellen mit Primär- und Fremdschlüsseln an. Beispielhaft die Haupttabelle materialien:

```
CREATE TABLE materialien (
  material_id INT AUTO INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  einheit VARCHAR(30) NOT NULL,
  mindestbestand INT NOT NULL DEFAULT 0,
  kategorie_id INT NOT NULL,
  FOREIGN KEY (kategorie_id)
    REFERENCES kategorien(kategorie_id)
);
```

Listing 1: CREATE-Statement der Tabelle materialien

Die Datenbankverbindung ist als Singleton umgesetzt, damit nur eine wiederverwendbare Verbindung besteht:

```
public class DBConnection {
    private static final String URL =
        "jdbc:mysql://127.0.0.1:3324/DOIT"
        + "?serverTimezone=Europe/Berlin";
    private static Connection connection;
    public static Connection getConnection() throws SQLException {
        if (connection == null || connection.isClosed())
            connection = DriverManager.getConnection(URL, "root", "1234");
        return connection;
    }
}
```

Listing 2: Singleton-Datenbankverbindung (DBConnection)

5.2 DAO-Schicht und Model-Klassen

Jede DAO-Klasse setzt die CRUD-Operationen um. Lesende Zugriffe liefern Listen über `findAll()`; schreibende Zugriffe nutzen PreparedStatements gegen SQL-Injection. Beispiel für das Anlegen eines Materials:

```
public Material create(Material material) throws SQLException {
    String sql = "INSERT INTO materialien (name, einheit, "
        + "mindestbestand, kategorie_id) VALUES (?, ?, ?, ?)";
    Connection conn = DBConnection.getConnection();
    try (PreparedStatement ps = conn.prepareStatement(sql,
        Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, material.getName());
        ps.setString(2, material.getEinheit());
        ps.setInt(3, material.getMindestbestand());
        ps.setInt(4, material.getKategorieId());
        ps.executeUpdate();
    }
    return material;
}
```

Listing 3: CRUD-Beispiel: create() in MaterialDAO

Damit der CRUD-Vertrag nicht in jeder Klasse neu beschrieben werden muss, definieren ihn alle DAO-Klassen über die gemeinsame Schnittstelle `GenericDAO<T>`. Das Interface enthält nur die vier Methodenköpfe `create`, `findAll`, `update` und `delete`; wer es implementiert, muss sie ausprogrammieren, sonst verweigert der Compiler die Übersetzung. Der Typ-Platzhalter `<T>` stammt aus den Java-Generics und steht immer für einen Referenztyp, also ein Objekt wie `Material` oder `Kategorie`; festgelegt wird er beim Implementieren. `delete(int id)` nutzt dagegen den Werttyp `int`, und `findAll()` gibt seine Ergebnisse als `List<T>` zurück. An dieser einen Stelle treffen damit Werttyp, Referenztyp und `Collection` zusammen – drei Konzepte, die sonst im Code verstreut auftauchen würden.

```
public interface GenericDAO<T> {
    T create(T obj) throws SQLException;
    List<T> findAll() throws SQLException;
    void update(T obj) throws SQLException;
    void delete(int id) throws SQLException;
}
```

```
}
```

Listing 4: Gemeinsame Schnittstelle GenericDAO<T>

In den sechs DAO-Klassen kennzeichnet die Annotation `@Override` jede dieser vier Methoden als Umsetzung des Schnittstellen-Vertrags; der Compiler prüft dadurch, dass die Signaturen exakt zur Schnittstelle passen.

Tabelle 7: Zuordnung der Klassen zu den MVC-Schichten

Schicht	Inhalt
Model	Material, Kategorie, Lieferant, Stationslager, Bestellung, Bestandsbewegung
View	JavaFX-FXML (main.fxml): Hauptfenster mit Registerkarten je Entität
Controller	MainController: steuert die UI-Logik, delegiert an die DAO-Schicht
DAO	MaterialDAO, KategorieDAO, LieferantDAO, StationslagerDAO, ...

5.3 Benutzeroberfläche und CRUD

Das Hauptfenster baut auf einem `TabPane` auf. Die Bestandsübersicht zeigt je Material und Lager den aus den Bewegungen berechneten Bestand – ermittelt per `JOIN` über `findBestandViews` – mit den Spalten Material, Einheit, Bestand, Mindestbestand und Status. Auf der Registerkarte Materialien werden die Stammdaten gepflegt (Name, Einheit, Mindestbestand, Kategorie-ID). Das Anlegen, Bearbeiten und Löschen läuft jeweils über einen Dialog; umgesetzt ist das vollständige CRUD für alle sechs Entitäten.

5.4 Such- und Filterfunktion

Die Bestandsübersicht lässt sich filtern. Suchfeld (Materialname) und Stationsauswahl wirken zusammen, angezeigt wird nur, was beide Bedingungen erfüllt. Ausgelöst wird die Filterung über die Eingabetaste oder die Schaltfläche „Suchen“:

```
private void anzeigeFiltern() {
    String suchtext = txtSuche.getText() == null
        ? "" : txtSuche.getText().trim().toLowerCase();
    Stationslager lager = cmbLager.getValue();
    ObservableList<BestandView> gefiltert =
        FXCollections.observableArrayList();
    for (BestandView v : alleBestaende) {
        boolean passtName = suchtext.isEmpty()
            || v.getMaterialName().toLowerCase().contains(suchtext);
        boolean passtLager = lager == null
            || v.getLagerId() == lager.getId();
        if (passtName && passtLager) gefiltert.add(v);
    }
    bestandListe.setAll(gefiltert);
}
```

Listing 5: Filterung der Bestandsübersicht nach Name und Station

5.5 Mindestbestand-Warnung und Lagerbuchungen

Liegt der Bestand auf oder unter dem Mindestbestand, färbt eine RowFactory die Zeile rot:

```
tabBestand.setRowFactory(tv -> new TableRow<BestandView>() {
    @Override protected void updateItem(BestandView item,
        boolean empty) {
        super.updateItem(item, empty);
        if (empty || item == null)    setStyle("");
        else if (item.isWarnung())
            setStyle("-fx-background-color: #FADBD8;");
        else setStyle("");
    }
}); // isWarnung(): bestand <= mindestbestand
```

Listing 6: Mindestbestand-Warnung über eine RowFactory

Bestandsänderungen laufen ausschließlich über Buchungen. Auf der Registerkarte Bestandsbewegungen lassen sich Ein- und Ausgänge anlegen, bearbeiten und löschen; eine Entnahme wird als AUSGANG, eine Umlagerung zwischen zwei Lagern als AUSGANG im Quell- und EINGANG im Ziellager gebucht. Der Status einer Bestellung – offen, geliefert oder storniert – beschreibt nur den Bestellvorgang selbst; den zugehörigen Wareneingang bucht der Nutzer anschließend als EINGANG.

6 Abnahme- und Testphase

6.1 Testfälle und Testergebnisse

Zur Qualitätssicherung wurden zehn Testfälle definiert, die alle Pflicht- und Zusatzanforderungen abdecken. Jeder davon wird manuell durchgeführt und das Ergebnis als bestanden oder fehlgeschlagen protokolliert. Da die Funktionen vollständig umgesetzt und die in Phase 6 gefundenen Fehler behoben sind, sollten alle Pflicht-Testfälle bestehen.

Tabelle 8: Definierte Testfälle (Auszug)

#	Testfall	Erwartetes Ergebnis	Prio
T1	Material anlegen	Erscheint in Tabelle, DB-Eintrag vorhanden	Hoch
T2	Material bearbeiten	Geänderte Werte in Tabelle und DB	Hoch
T3	Material löschen	Aus Tabelle und DB entfernt	Hoch
T4	Mindestbestand-Warnung	Zeile rot bei Bestand $\leq$ Mindestbestand	Hoch
T5	Suche nach Name	Tabelle filtert live beim Tippen	Mittel
T6	Filter nach Station	Nur Material der gewählten Station	Mittel
T7	AUSGANG buchen	Bestand sinkt, Buchung in Historie	Hoch
T8	EINGANG buchen	Bestand steigt, Buchung in Historie	Mittel
T9	Pflichtfeld leer	Fehlermeldung, kein DB-Insert	Hoch

Während der Umsetzung traten mehrere Fehler auf. In der BestandsbewegungDAO war ein Primärschlüssel falsch benannt, in der DBConnection stimmten Datenbankname und Port nicht,

und einmal wurde ein ENUM-Wert (TRANSFER) verwendet, den das Schema gar nicht vorsah. Lokalisiert habe ich die Fehler jeweils über die Fehlermeldung; Ursache und Lösung sind im Projekttagebuch festgehalten.

6.2 Soll-Ist-Vergleich

Tabelle 9: Soll-Ist-Vergleich der Anforderungen

Anforderung	Umsetzung	Status
MySQL $\geq$ 5 Tabellen	6 Tabellen umgesetzt	erfüllt
CRUD-Operationen	Vollständig für alle 6 Entitäten	erfüllt
MVC-Architektur	Model / View / Controller + DAO	erfüllt
Such-/Filterfunktion	Suche nach Name, Filter nach Station	erfüllt
Mindestbestand-Warnung	Rote Zeilenfärbung (RowFactory)	erfüllt
Buchungshistorie	Tabelle bestandsbewegungen	erfüllt

6.3 Abnahme und Übergabe

Die formale Abnahme erfolgt durch Frau Cramer am Ende von Block 2 (KW 26, bis 26.06.2026). Grundlage ist die vollständige Umsetzung der Pflichtenforderungen: lauffähige Anwendung, stabile Datenbankverbindung, korrekte Such- und Filterfunktion, sichtbare Mindestbestand-Warnung sowie eine vollständige Dokumentation. Übergeben werden der Quellcode (Maven-Projekt, Git), das Datenbankskript (Anhang B), das Benutzerhandbuch (Anhang C), das ER-Diagramm (Anhang A) und die Projektdokumentation als PDF (Moodle). Bekannte Einschränkungen der aktuellen Version: keine Benutzerverwaltung mit Login, keine automatische Nachbestellung (das System warnt nur), keine Netzwerkfunktion und kein Export nach PDF oder Excel.

7 Dokumentation

7.1 Benutzerhandbuch

Für die Endnutzer entstand ein ausführliches Benutzerhandbuch. Es führt Schritt für Schritt durch Programmstart, Navigation und die einzelnen Funktionen – vom Anlegen eines Materials über Suche und Filter bis zur Buchungshistorie – und schließt mit einem FAQ-Teil zur Fehlerbehebung. Das vollständige Handbuch steht in Anhang C.

7.2 Entwicklerdokumentation

Damit sich das Projekt technisch nachvollziehen lässt, ist der Quellcode kommentiert und nach dem MVC- und DAO-Muster gegliedert; Klassen- und Methodennamen sind sprechend

gewählt. Eine README erläutert Aufbau und Start, das Datenbankskript legt Schema und Testdaten an, und im Projekttagbuch sind die Entscheidungen samt Problemen und Lösungen dokumentiert. Wer die Anwendung später weiterentwickelt, findet so einen Einstieg.

8 Fazit

8.1 Soll-Ist-Vergleich der Projektziele

Die Kernziele sind erreicht. Alle vier Pflichtenforderungen wurden umgesetzt: die sechs Tabellen, das vollständige CRUD, die MVC-Architektur mit DAO-Schicht sowie Suche, Filter und Mindestbestand-Warnung laufen wie vorgesehen. Auch der Zeitplan ist aufgegangen – der eingeplante Puffer verschaffte in der Test- und Dokumentationsphase etwas Luft. Insgesamt bin ich mit dem Ergebnis zufrieden.

8.2 Lessons Learned

Das DAO-Muster, die Singleton-Verbindung und den Aufbau der Model-Klassen kannte ich schon aus dem Unterricht, dem Dino-Park-Beispiel von Frau Cramer. Die JavaFX-Themen musste ich mir dagegen selbst erarbeiten – TableView mit ObservableList, die gefilterte Bestandssicht, Dialoge, das TabPane und die Zeilenfärbung über eine RowFactory. Auf SQL-Ebene war für mich die wichtigste Erkenntnis, den Bestand nicht zu speichern, sondern per Aggregation aus den Bewegungen zu berechnen. Und im Umgang mit Fehlern hat sich ein festes Vorgehen bewährt: erst die Meldung lesen, dann die Ursache eingrenzen, am Ende die Lösung ins Tagebuch schreiben.

Eine größere Umstrukturierung kam erst spät dazu. Mir fiel auf, dass alle sechs DAO-Klassen dieselben vier Methoden enthielten – also habe ich diese in der Schnittstelle GenericDAO<T> zusammengefasst und nachträglich in jede DAO eingezogen. Der gemeinsame Aufbau steht jetzt nur noch an einer Stelle, und der Compiler stellt sicher, dass sich jede Klasse daran hält. Am Verhalten des Programms hat sich nichts geändert.

Ähnlich bin ich später beim MainController vorgegangen, der mit über 600 Zeilen für alle sieben Reiter kaum noch zu überblicken war. Inzwischen hat jeder Reiter seine eigene kleine Klasse, und der wiederkehrende Ablauf liegt nur einmal in einer Basisklasse. Funktional ist alles beim Alten geblieben, der Code ist aber kürzer und leichter zu lesen.

8.3 Ausblick

Für einen produktiven Einsatz wären noch einige Erweiterungen sinnvoll – eine Benutzerverwaltung mit Login und Rollen, eine Backup-Funktion und ein Export nach PDF

und Excel. Beim nächsten Mal würde ich die Testfälle früher und begleitend zur Entwicklung schreiben und der Oberfläche von Anfang an ein einheitliches CSS-Stylesheet geben. Im gegebenen Zeitrahmen hatten diese Punkte bewusst keine Priorität.

Abgabeerklärung

Ich erkläre hiermit, dass ich diese Projektdokumentation selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Ort, Datum	Unterschrift (Sascha Schulz)
------------	------------------------------

Quellenverzeichnis

- [1] Oracle Corporation: Java SE 17 Documentation. Oracle, 2021. URL: <https://docs.oracle.com/en/java/javase/17/> (Abruf: 04.05.2026)
- [2] Oracle / OpenJFX: JavaFX 21 API Documentation. 2021. URL: <https://openjfx.io/javadoc/21/> (Abruf: 04.05.2026)
- [3] Oracle / MySQL: MySQL 8.0 Reference Manual. 2023. URL: <https://dev.mysql.com/doc/refman/8.0/en/> (Abruf: 04.05.2026)
- [4] SRH Berufliche Rehabilitation GmbH: DOIT-Leitfaden, Gruppe 2551. Unveröffentlichtes internes Dokument, Heidelberg, 2024.
- [5] Gamma, E. / Helm, R. / Johnson, R. / Vlissides, J.: Design Patterns. Addison-Wesley, Boston, 1994.
- [6] Oracle / OpenJFX: JavaFX 21 – TableView (API). URL: <https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TableView.html> (Abruf: 07.06.2026)
- [7] Oracle / OpenJFX: JavaFX 21 – ObservableList (API). URL: <https://openjfx.io/javadoc/21/javafx.base/javafx/collections/ObservableList.html> (Abruf: 07.06.2026)
- [8] Oracle / OpenJFX: JavaFX 21 – TabPane (API). URL: <https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TabPane.html> (Abruf: 07.06.2026)
- [9] Oracle / OpenJFX: JavaFX 21 – Dialog (API). URL: <https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/Dialog.html> (Abruf: 07.06.2026)
- [10] Oracle: Introduction to FXML (JavaFX-Tutorial). URL: https://docs.oracle.com/javase/8/javafx/fxml-tutorial/get_started.htm (Abruf: 07.06.2026)
- [11] Apache Software Foundation: Maven – Getting Started Guide. URL: <https://maven.apache.org/guides/getting-started/> (Abruf: 07.06.2026)
- [12] Oracle: MySQL Connector/J Developer Guide. URL: <https://dev.mysql.com/doc/connector-j/en/> (Abruf: 07.06.2026)
- [13] Oracle: Java SE 8 API – java.sql.PreparedStatement. URL: <https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html> (Abruf: 07.06.2026)

Anhang

Anhang	Titel	Beschreibung
A	ER-Diagramm	Datenmodell der 6 Tabellen
B	SQL-Datenbankskript	CREATE-Statements aller Tabellen
C	Benutzerhandbuch	Vollständiges, detailliertes Handbuch
D	Wochenplan	Wochenplan_Sascha_Schulz.xlsx
E	Projektantrag	Projektantrag_Sascha_Schulz_v2.docx
F	Glossar	Fachbegriffe des Projekts (Java, JavaFX, JDBC, SQL)

Anhang A – ER-Diagramm

Das ER-Diagramm zeigt alle sechs Tabellen mit Entitäten, Attributen, Primär- und Fremdschlüsseln sowie den Kardinalitäten (1:N). Der Bestand wird nicht gespeichert, sondern aus bestandsbewegungen berechnet.

ER-Diagramm: Materialverwaltung Krankenhaus (6 Tabellen, 3NF)

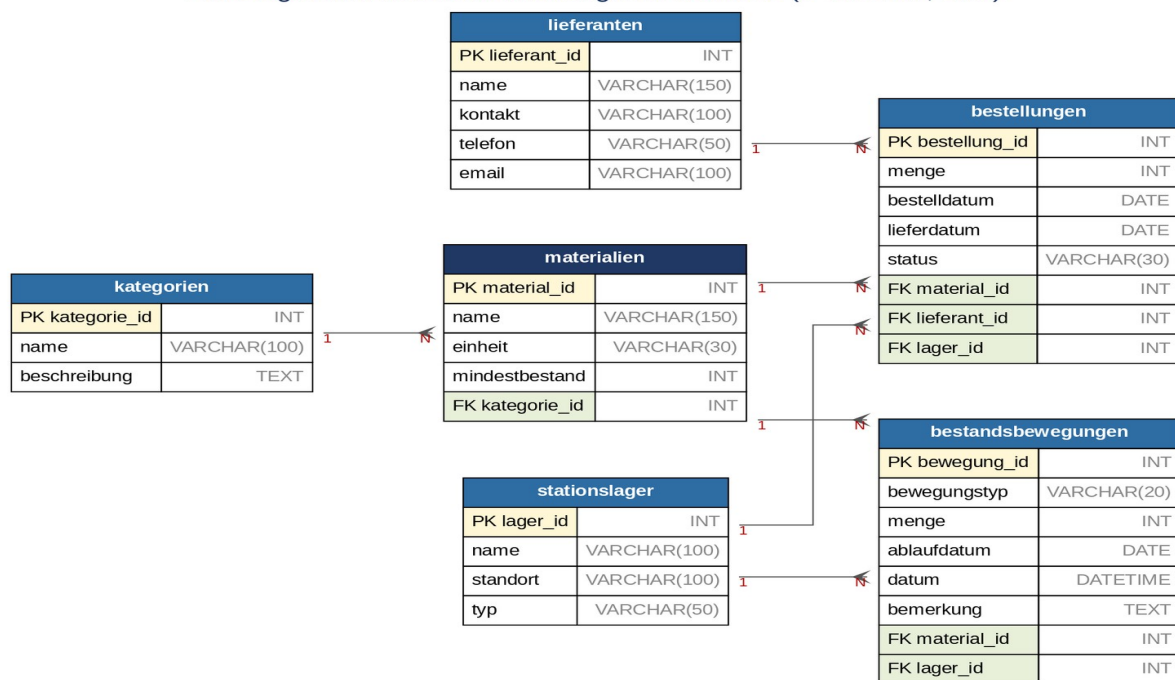


Abbildung 5: ER-Diagramm der Datenbank (Anhang A)

Anhang B – SQL-Datenbankskript

Datenbank DOIT (utf8mb4). Nachfolgend die CREATE-Statements aller sechs Tabellen; das vollständige Skript inkl. Testdaten liegt in testdaten_krankenhaus_lager.sql bei.

```

CREATE DATABASE DOIT
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
  
```

```

USE DOIT;

CREATE TABLE kategorien (
  kategorie_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, beschreibung TEXT );

CREATE TABLE stationslager (
  lager_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, standort VARCHAR(100), typ VARCHAR(50) );

CREATE TABLE lieferanten (
  lieferant_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, kontakt VARCHAR(100),
  telefon VARCHAR(50), email VARCHAR(100) );

CREATE TABLE materialien (
  material_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, einheit VARCHAR(30) NOT NULL,
  mindestbestand INT NOT NULL DEFAULT 0, kategorie_id INT NOT NULL,
  FOREIGN KEY (kategorie_id) REFERENCES kategorien(kategorie_id) );

CREATE TABLE bestellungen (
  bestellung_id INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL, lieferant_id INT NOT NULL,
  lager_id INT NOT NULL, menge INT NOT NULL,
  bestelldatum DATE NOT NULL, lieferdatum DATE,
  status VARCHAR(50) NOT NULL DEFAULT 'offen',
  FOREIGN KEY (material_id) REFERENCES materialien(material_id),
  FOREIGN KEY (lieferant_id) REFERENCES lieferanten(lieferant_id),
  FOREIGN KEY (lager_id) REFERENCES stationslager(lager_id) );

CREATE TABLE bestandsbewegungen (
  bewegung_id INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL, lager_id INT NOT NULL,
  bewegungstyp ENUM('EINGANG','AUSGANG') NOT NULL,
  menge INT NOT NULL, ablaufdatum DATE,
  datum DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, bemerkung TEXT,
  FOREIGN KEY (material_id) REFERENCES materialien(material_id),
  FOREIGN KEY (lager_id) REFERENCES stationslager(lager_id) );

```

Listing 7: Datenbankschema – CREATE-Statements aller sechs Tabellen

Anhang C – Benutzerhandbuch

Materialverwaltung für medizinisches Verbrauchsmaterial – Krankenhaus-Edition (JavaFX + MySQL). Version 1.0. Zielgruppe: Pflege- und Stationspersonal sowie Verwaltung. Dieses Handbuch führt Schritt für Schritt durch alle Funktionen.

C.1 Systemvoraussetzungen

Komponente	Anforderung
Betriebssystem	Windows 10 oder 11 (64-Bit)
Java Runtime	Java 17 oder neuer (JRE/JDK)
Datenbankserver	MySQL 8.x – Dienst muss laufen
Datenbank	DOIT – muss angelegt sein
Auflösung / RAM	min. 1280 × 768; min. 4 GB RAM

C.2 Programmstart

So starten Sie die Anwendung:

1. Prüfen Sie, ob der MySQL-Dienst läuft (Windows-Dienste > „MySQL“ > Status „Wird ausgeführt“).
2. Starten Sie das Programm per Doppelklick auf KrankenhausLager.jar oder über Main.java in der IDE.
3. Das Hauptfenster öffnet sich und zeigt automatisch die Registerkarte „Bestandsübersicht“.
4. Erscheint die Meldung „Datenbankverbindung fehlgeschlagen“, starten Sie den MySQL-Dienst und prüfen die Zugangsdaten in DBConnection.java.

Das Fenster gliedert sich in die Tab-Leiste (oben), die Such- und Filterleiste, die Tabelle (Mitte) und die Schaltflächen Hinzufügen, Bearbeiten, Löschen und Aktualisieren (unten).

Registerkarte	Funktion
Bestandsübersicht	Startansicht: Bestand je Material und Lager, Suche, Stationsfilter, Warnung
Materialien	Stammdaten der Materialien verwalten (Kern)
Kategorien	Materialgruppen verwalten
Stationslager	Stationen und Lager verwalten
Lieferanten	Lieferantenstammdaten (optional)
Bestellungen	Nachbestellungen erfassen und verfolgen (optional)
Bestandsbewegungen	Ein- und Ausgänge manuell buchen und nachvollziehen

C.3 *Material anlegen*

5. Öffnen Sie die Registerkarte „Materialien“.
6. Klicken Sie unten auf „Hinzufügen“. Es öffnet sich ein Dialogfenster.
7. Feld „Name“: Materialbezeichnung eingeben, z. B. Einmalhandschuhe M.
8. Feld „Einheit“: Mengeneinheit eingeben, z. B. Stk, Pkg oder Fl.
9. Feld „Mindestbestand“: Zahl eingeben, ab welcher gewarnt wird, z. B. 100.
10. Feld „Kategorie-ID“: die Nummer der gewünschten Kategorie eingeben, z. B. 1 (die IDs sehen Sie auf der Registerkarte Kategorien).
11. Auf „Speichern“ klicken. Das Material erscheint sofort in der Tabelle.

Hinweis: Leere Pflichtfelder werden rot markiert; der Datensatz wird dann nicht gespeichert. Ein Material ist eine reine Stammdatei – der Bestand ergibt sich aus den Buchungen, nicht aus dem Material selbst.

C.4 *Material bearbeiten und löschen*

Bearbeiten: Zeile in der Tabelle anklicken › „Bearbeiten“ (oder Doppelklick) › Werte im Dialog ändern › „Speichern“. Die Tabelle aktualisiert sich automatisch.

Löschen: Zeile anklicken › „Löschen“ › Sicherheitsabfrage mit „Ja“ bestätigen. Achtung: Gelöschte Datensätze lassen sich nicht wiederherstellen. Lässt sich ein Material nicht löschen, bestehen noch abhängige Bestellungen oder Buchungen, die zuerst entfernt werden müssen.

C.5 *Suchen und Filtern*

12. Öffnen Sie die Registerkarte „Bestandsübersicht“.
13. Tippen Sie im Suchfeld einen Materialnamen (z. B. Handschuh) und drücken Sie Enter oder die Schaltfläche „Suchen“. Groß-/Kleinschreibung spielt keine Rolle.
14. Wählen Sie im Dropdown „Station“ eine Station, um nur deren Bestände zu sehen.
15. Such- und Stationsfilter wirken zusammen. „Zurücksetzen“ leert beide Filter, „Aktualisieren“ lädt die Daten neu.

C.6 *Mindestbestand-Warnung verstehen*

In der Bestandsübersicht wird die gesamte Zeile rot eingefärbt, sobald der Bestand auf oder unter dem Mindestbestand liegt. Das ist das Signal, dieses Material dringend nachzubestellen. Die Färbung passt sich automatisch an, sobald durch eine Buchung wieder genug Bestand vorhanden ist.

C.7 Kategorien, Stationslager und Lieferanten

Diese Stammdaten werden auf den gleichnamigen Registerkarten gepflegt – jeweils mit denselben Schaltflächen Hinzufügen, Bearbeiten und Löschen. Jedes Material muss einer Kategorie zugeordnet sein; eine Kategorie lässt sich nur löschen, wenn ihr kein Material mehr zugeordnet ist. Stationslager bilden Stationen und Lager ab (z. B. Station 1, OP, ICU, Zentrallager). Lieferanten (optional) speichern die Stammdaten der Bezugsfirmen.

C.8 Bestellungen erfassen

16. Registerkarte „Bestellungen“ öffnen › „Hinzufügen“.
17. Material und Lieferant im Dropdown wählen.
18. Menge und Bestelldatum eingeben. Der Status wird automatisch auf „offen“ gesetzt.
19. Auf „Speichern“ klicken – die Bestellung erscheint in der Liste.
20. Sobald die Lieferung eintrifft: Bestellung auswählen und den Status auf geliefert setzen, dann speichern.

Hinweis: Das Setzen des Status verändert den Bestand nicht automatisch. Den Wareneingang buchen Sie anschließend als EINGANG auf der Registerkarte Bestandsbewegungen (siehe C.9). Statuswerte: offen, geliefert, storniert.

C.9 Buchungen (Ein- und Ausgang)

Auf der Registerkarte Bestandsbewegungen werden alle Ein- und Ausgänge gebucht und protokolliert. So buchen Sie eine Bewegung:

21. Registerkarte Bestandsbewegungen öffnen, dann Anlegen.
22. Material und Stationslager wählen.
23. Bewegungstyp EINGANG (Zugang) oder AUSGANG (Entnahme) auswählen.
24. Menge eingeben und optional eine Bemerkung (z. B. Entnahme OP).
25. Auf „Speichern“ klicken. Datum und Uhrzeit werden automatisch gesetzt; der Bestand passt sich an.

Eine Umlagerung zwischen zwei Lagern erfassen Sie als AUSGANG im Quelllager und EINGANG im Ziellager. Nur durch konsequentes Buchen bleibt der Bestand lückenlos nachvollziehbar.

C.10 FAQ und Fehlerbehebung

Frage / Meldung	Lösung
Programm startet nicht	Java 17 installiert? (java -version). MySQL-Dienst aktiv? Zugangsdaten

Frage / Meldung	Lösung
	prüfen.
Datenbankverbindung fehlgeschlagen	MySQL-Dienst in den Windows-Diensten starten.
Access denied for user	Falsches Passwort - Zugangsdaten in DBConnection.java prüfen.
Zeile ist rot markiert	Bestand kleiner oder gleich Mindestbestand - Material nachbestellen.
Material lässt sich nicht löschen	Erst abhängige Bestellungen/Buchungen entfernen.
Gelöschte Daten zurückholen?	Nicht möglich - mit Bedacht löschen.

Support: IT-Systemadministrator vor Ort oder Entwickler Sascha Schulz (Gruppe 2551).
Benutzerhandbuch Version 1.0 – SRH Berufliche Rehabilitation, WI-Ausbildung 2026.

Anhang D – Wochenplan

Der vollständige Wochenplan liegt der Abgabe als separate Datei Wochenplan_Sascha_Schulz.xlsx bei. Er enthält die wochenweise Planung der sieben Projektphasen über die sechs Projektwochen (04.05.2026 – 26.06.2026).

Anhang E – Projektantrag

Der genehmigte Projektantrag liegt der Abgabe als separate Datei Projektantrag_Sascha_Schulz_v2.docx bei. Er dokumentiert die ursprüngliche Projektplanung mit Phasen und Stundenansätzen (180 h).

Anhang F – Glossar

Das folgende Glossar erläutert die im Projekt verwendeten Fachbegriffe der Java-Standardbibliothek (JDBC, Collections, JavaFX) sowie zentrale Begriffe der Anwendung. Es umfasst 90 Begriffe in alphabetischer Reihenfolge.

Begriff	Kategorie	Erläuterung
ActionEvent	JavaFX Events	Event beim Klick auf Button oder ähnliche Komponenten. Parameter der EventHandler<ActionEvent>.
Alert	JavaFX UI	JavaFX-Dialogfenster für Systemmeldungen. Typen: INFORMATION, WARNING, ERROR, CONFIRMATION; wird mit alert.showAndWait() blockierend angezeigt.
ArrayList	Collections	Dynamisches Array mit

		variabler Größe. Häufigste Implementierung von List. In DAO-Klassen: new ArrayList<Material>() für Rückgabewerte von findAll(). Statische Hilfsmethoden für Arrays: sort(), asList(), binarySearch() etc.
Arrays	java.util	
BorderPane	JavaFX Layout	Layout-Manager mit 5 Bereichen: top, bottom, left, right, center. Typisch für Fenster mit Menü oben und Inhaltsbereich unten.
BufferedReader / BufferedWriter	java.io	Gepufferte Reader/Writer für effiziente Dateiverarbeitung. Typisch: readLine(), write(String).
Button	JavaFX UI	Schaltfläche. Reagiert auf Klicks via setOnAction(EventHandler<ActionEvent>).
ButtonType	JavaFX UI	Repräsentiert eine Schaltfläche in einem JavaFX-Dialog (z. B. ButtonType.OK, ButtonType.CANCEL); ermöglicht Auswertung der Nutzerwahl via dialog.showAndWait().
ChangeListener	JavaFX Listeners	Listener für Eigenschafts-Änderungen. Beispiel: tableView.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) -> {...}).
Cloneable	java.lang	Marker-Interface für Objektkopien.

		Implementierung von clone() möglich.
Collections	java.util	Statische Hilfsmethoden für Collections: sort(), reverse(), shuffle(), binarySearch() etc.
ComboBox	JavaFX UI	Dropdown-Liste für Auswahl aus vordefinierten Werten. Typisch für Kategorien- oder Lieferanten-Auswahl in Masken.
Comparable	java.lang	Interface für natürliche Ordnung (implementiert in Klasse). Definiert compareTo().
Comparator	java.util	Interface für externe Sortierlogik (separate Klasse). Beispiel: list.sort(Comparator.comparing(Material::getBestand)).
Connection	JDBC (java.sql)	Schnittstelle zur Datenbank-Verbindung. Verwaltet die Kommunikation zwischen Java-Anwendung und MySQL-Datenbank. Wird mittels DriverManager.getConnection() erzeugt.
Consumer	java.util.function	Functional Interface für Operationen ohne Rückgabewert. Typisch: forEach(item -> System.out.println(item)).
DatePicker	JavaFX UI	JavaFX-Steuererelement zur grafischen Datumsauswahl; liefert ein LocalDate-Objekt über getValue() und unterstützt Validierung und Formatierung.
DateTimeFormatter	java.time	Formatter für Datums-/Zeit-

Dialog

JavaFX UI

Strings. Beispiel: `DateTime
Formatter.ofPattern("dd.MM.y
yyy").form
at(LocalDate.now()).`
Generisches JavaFX-
Dialogfenster mit
konfigurierbarem Inhalt und
Rückgabety; Basis für Alert,
ChoiceDialog und
TextInputDialog.

DriverManager

JDBC (java.sql)

Zentrale Klasse zur Verwaltung
von JDBC-Treibern.

Hier ist der genaue Ablauf:

1. DriverManager

`r.getConnection(URL, USER,
PASSWORD)` wird aufgerufen.

2. URL, USER, PASSWORD
sind statische Klassenvariablen
der DBConnection-Klasse.

3. DriverManager bekommt die
Zugangsdaten über die
Parameter — er "kennt" die
DBConnection-Klasse nicht.

4. Die Parameter werden an
`getConnection()` übergeben,
nicht von DriverManager selbst
gefunden.

Die DBConnection-Klasse
definiert die Zugangsdaten und
gibt sie als Parameter weiter.

DriverManager selbst kennt
keine Zugangsdaten.

Exception / RuntimeException java.lang

Exception-Hierarchie:

Exception = geprüfte

Exceptions (müssen behandelt
werden), RuntimeException =

ungeprüfte Exceptions

EventHandler	JavaFX Events	(optional). Functional Interface für Event- Behandlung. Beispiel: button.setOnAction(e -> handleClick());.
File	java.io	Repräsentiert Da teien/Verzeichnisse. Ermöglicht Lesen von Dateipfaden, Prüfung auf Existenz etc.
Float / Integer / Long / Double / Boolean	java.lang	Wrapper-Klassen für primitive Typen. Ermöglichen null- Werte und Co llections<Integer> etc. Automatisches Boxing/Unboxing seit Java 5.
Function<T,R>	java.util.function	Functional Interface für Transformationen ($T \rightarrow R$). Beispiel: map(m -> m.getName()).
FXML	JavaFX UI Definition	XML-Format für UI-Layouts in JavaFX. Alternative zu programmatischer Kom ponenten-Erstellung. .fxml- Dateien werden mit FXMLLoader geladen.
FXMLLoader	JavaFX	Loader für FXML-Dateien (XML-basierte UI- Definitionen). Erzeugt Szenegraph aus .fxml-Datei: FXMLLoader loader = new FX MLLoader(getClass(). getResource(...));.
GridPane	JavaFX Layout	Raster-Layout mit Zeilen und Spalten. Ideal für Formulare: Label in Spalte 0, TextField in Spalte 1. Kinder werden in

		einem Zeilen-Spalten-Raster angeordnet; präzise Positionierung über <code>addRow()</code> oder <code>setRowIndex()</code>/<code>setColumnIndex()</code>.
<code>HashMap<K,V></code>	Collections	Hash-basierte Map für Schlüssel-Wert-Paare. Erlaubt schnelle $O(1)$ Lookups. Beispiel: <code>HashMap<Integer, Material></code> für ID-basierte Caches.
<code>HBox / VBox</code>	JavaFX Layout	Horizontale / vertikale Anordnung von Komponenten. <code>HBox</code> stapelt horizontal, <code>VBox</code> vertikal mit konfigurierbarem <code>Spacing</code> und <code>Alignment</code>.
<code>IOException</code>	<code>java.io</code>	Exception bei I/O-Fehlern (Datei nicht gefunden, Les-/Schreibfehler). Geprüfte Exception (muss behandelt oder deklariert werden).
<code>Iterable</code>	<code>java.lang</code>	Interface für Collections, die in for-each-Schleifen iteriert werden können. Definiert <code>iterator()</code>.
<code>Iterator</code>	<code>java.util</code>	Interface für explizite Iteration mit <code>hasNext()</code> / <code>next()</code>.
<code>Label</code>	JavaFX UI	Statischer oder dynamischer Text. Keine Benutzereingabe.
<code>List</code>	Collections	Generische Interface für geordnete Sammlungen. Implementiert von <code>ArrayList</code>, <code>LinkedList</code> etc. Typisch: <code>List<Material></code> für Collections von Objekten.
<code>LocalDate / LocalDateTime</code>	<code>java.time</code>	Moderne Datums-/Zeitklassen ohne Timezone-Probleme (seit

Objects	java.util	Java 8). LocalDate: nur Datum, LocalDateTime: Datum + Zeit. Hilfsmethoden für Null-Checks (seit Java 7): Objects.requireNonNull(), Objects.equals() etc.
ObservableList	JavaFX	Spezielle List für JavaFX, die Änderungen meldet. Wird in TableView und ComboBox gebunden. Erzeugt mit FXCollections.observableArrayList().
ObservableValue	JavaFX Properties	Basis-Interface für beobachtbare Eigenschaften. Beispiele: selectedItemProperty(), textProperty(), valueProperty().
Optional	java.util	Wrapper für Werte, die null sein könnten (seit Java 8). Sichere Alternative zu null-Checks. Beispiel: Optional<Material> m = dao.findById(1); Vermeidet NullPointerException beim Auswerten von Dialog-Ergebnissen.
Platform.runLater()	JavaFX Threading	Führt Code im JavaFX-Thread aus. Notwendig bei Änderungen an UI-Komponenten von außerhalb des FX-Threads.
Predicate	java.util.function	Functional Interface für Bedingungen (boolean). Typisch in filter(): stream.filter(m -> m.getBestand() > 0).
PreparedStatement	JDBC (java.sql)	Vorkompiliertes SQL-

		Statement mit Platzhaltern (?). Verhindert SQL-Injection durch parametrisierte Abfragen. Parameter werden mit setInt(), setString() etc. gesetzt.
Properties	java.util	Schlüssel-Wert-Speicher für Konfigurationen. Oft in .properties-Dateien gespeichert (DB-Credentials, UI-Texte).
ResultSet	JDBC (java.sql)	Tabellarische Ergebnismenge von SQL-Abfragen. Wird mit executeQuery() erzeugt. Ermöglicht zeilenweise Iteration mit next() und Datenzugriff mit getInt(), getString() etc.
Runnable	java.lang	Functional Interface mit run()-Methode. Basis für Threads und asynchrone Operationen.
Scanner	java.util	Tokenizing Reader für Eingaben (Datei, Console). Beispiel: new Scanner(file).useDelimiter("\n").
Scene	JavaFX Core	Grafische Szene in JavaFX. Enthält Szenegraph (Komponenten-Hierarchie). Wird in Stage angezeigt.
Serializable	java.io	Marker-Interface für Objekt-Persistierung (Serialisierung).
SQLException	JDBC (java.sql)	Exception, die bei Datenbank-Fehlern geworfen wird (Verbindungsprobleme, SQL-Fehler, Treiber-Fehler). Muss in throws-Klausel deklariert oder mit try-catch behandelt

Stage	JavaFX Core	werden. Fenster in JavaFX (Entsprechung zu JFrame in Swing). Enthält Scene. start(Stage stage) ist Einstiegspunkt jeder JavaFX-Anwendung.
Stream	java.util.stream	Funktionale Verarbeitung von Collections (seit Java 8). Beispiel: <code>li.stream().filter(m -> m.getBestand() < 10).collect(Collectors.toList());</code>
String	java.lang	Unveränderliche Textklasse. Wichtigste Methoden: <code>length()</code> , <code>substring()</code> , <code>split()</code> , <code>contains()</code> , <code>replace()</code> .
Tab	JavaFX UI	Einzelner Reiter innerhalb eines <code>TabPane</code> in JavaFX; enthält Titel, optionales Icon und beliebigen Node-Inhalt; Lazy-Loading via <code>setOnSelectionChanged()</code> .
TabPane	JavaFX UI	JavaFX-Container, der mehrere Tab-Objekte als Register-Reiter darstellt; der sichtbare Tab wird über <code>getSelectedTab()</code> beobachtet.
TableColumn<S,T>	JavaFX UI	Einzelne Spalte in <code>TableView</code> . Bindet Objekt-Eigenschaften via <code>setCellValueFactory()</code> an Anzeige.
TableView	JavaFX UI	JavaFX-Komponente zur tabellarischen Darstellung von Daten. Zeigt

		ObservableList<T> mit Spalten (TableColumn). Typisch für Materialien, Bestellungen, Bestände.
Task	JavaFX Concurrency	Asynchrone Task für lange Operationen ohne UI-Blockade. Wird in separaten Thread ausgeführt.
TextField	JavaFX UI	Einzeiliges Eingabefeld. getText() liest Wert, setText() setzt ihn.
Thread	java.lang	Basis-Klasse für Multithreading. Task ist moderne Alternative für JavaFX.
Throwable	java.lang	Basis-Klasse aller Exceptions und Errors.
@Override	Annotation (java.lang)	Annotation, die dem Compiler mitteilt, dass die folgende Methode eine Methode der Elternklasse überschreibt. Hier ist der genaue Ablauf: 1. In MainController.java steht @Override über initialize() — JavaFX ruft diese Methode automatisch nach dem Laden von main.fxml auf. 2. In den Model-Klassen (Material, Kategorie, Lieferant, Bestandsbewegung) steht @Override über toString() — so gibt z.B. Kategorie.toString() den Kategorienamen zurück, der in ComboBoxen angezeigt wird. 3. Fehlt @Override und der Methodenname ist falsch

AlertType

JavaFX UI

geschrieben, schreibt man eine neue Methode statt die bestehende zu überschreiben — der Compiler warnt dann nicht. IntelliJ zeigt einen Pfeil nach oben im Gutter neben jeder @Override-Methode.

Aufzählung innerhalb der JavaFX-Klasse Alert, die das Icon und den Standardtitel des Dialogs bestimmt.

Hier ist der genaue Ablauf:

1. new Alert
Alert.AlertType.ERROR erzeugt einen Dialog mit rotem X-Icon — wird in MainController.zeigeFehler() bei Datenbankfehlern verwendet.
2. AlertType.WARNING zeigt ein gelbes Dreieck-Icon — wird in zeigeWarnung() für Mindestbestand-Unterschreitungen verwendet.
3. AlertType.INFORMATION zeigt ein blaues i-Icon — für Bestätigungsmeldungen nach erfolgreichen Operationen.

Der Typ wird als Parameter an den Alert-Konstruktor übergeben, nicht nachträglich gesetzt.

AnchorPane

JavaFX Layout

JavaFX-Layout-Container, bei dem Kinder-Elemente an einen oder mehrere Ränder (top, right, bottom, left) verankert werden können.

Hier ist der Zusammenhang im

Application

JavaFX Basisklasse

Projekt:

1. main.fxml verwendet eine VBox als Wurzelement — AnchorPane ist eine mögliche Alternative.
 2. Beide sind Unterklassen von Parent: Parent root = loader.load() in Main.start() akzeptiert jeden Container.
 3. Die Wahl des Containers bestimmt, wie enthaltene Elemente positioniert werden. AnchorPane eignet sich für absolute Positionierung; VBox und HBox für sequenzielle Anordnung.
- JavaFX-Basisklasse, von der Main.java im Projekt erbt.
- Hier ist der genaue Ablauf:
1. public class Main extends Application — durch extends Application erbt Main den gesamten JavaFX-Lebenszyklus.
 2. Launcher.main() ruft Main.launch(args) auf — launch() ist eine statische Methode von Application, die den JavaFX-Thread startet.
 3. JavaFX ruft danach automatisch start(Stage primaryStage) auf — diese Methode ist in Main.java implementiert und lädt main.fxml.
- Application ist abstract — start() muss zwingend

ChoiceDialog

JavaFX UI

implementiert werden, sonst kompiliert das Projekt nicht. Fertige JavaFX-Dialog-Variante für die Auswahl aus einer vordefinierten Liste. Hier ist der genaue Ablauf:

1. `new ChoiceDialog<T>(defaultValue, choices)` erzeugt einen Dialog mit einer Auswahl-ComboBox.
2. `showAndWait()` blockiert den JavaFX-Thread, bis der Benutzer eine Auswahl trifft.
3. `Optional<T> result = dialog.showAndWait(); result.ifPresent(wert -> {...})` wertet die Auswahl sicher aus.

Im Projekt werden Auswahloperationen über direkte ComboBox-Steuer-elemente in `main.fxml` realisiert — `ChoiceDialog` ist die Alternative für Auswahloperationen in separaten Popup-Fenstern.

`ClassNotFoundException`

`java.lang (Exception)`

Exception, die auftritt, wenn Java eine Klasse nicht laden kann.

Hier ist der genaue Ablauf im Projekt:

1. `FXMLLoader.load()` liest das Attribut `fx:controller` aus `main.fxml`: z.B. `fx:controller="de.doit.controller.MainController"`.
2. Java versucht, diese Klasse zu laden — schlägt das fehl

		(Tippfehler, falsches Package), wird eine <code>ClassNotFoundException</code> geworfen. 3. Der <code>FXMLLoader</code> verpackt sie als <code>caused by</code>-Ursache innerhalb einer <code>LoadException</code>. Beim Lesen des Fehlers: immer zuerst den <code>Exception</code>-Typ, dann die <code>caused by</code>-Zeile lesen — dort steht der tatsächliche Fehler.
<code>CommunicationsException</code>	MySQL JDBC (Exception)	Exception des MySQL JDBC- Treibers bei fehlgeschlagener Netzwerkverbindung zur Datenbank. Hier ist der genaue Ablauf: 1. <code>DriverManager.getConnection("jdbc:mysql://localhost:3306/lagerverwaltung", USER, PASSWORD)</code> versucht eine TCP-Verbindung zu Port 3306. 2. Der MySQL-Server antwortet nicht — nach dem Timeout wirft der JDBC-Treiber eine <code>CommunicationsException</code>. 3. Die Exception-Meldung lautet: "Communications link failure — The driver has not received any packets from the server." Lösung: MySQL-Dienst starten (XAMPP, Workbench oder Windows-Dienste). Die Exception erscheint immer in

DataSource

IntelliJ / java.sql

DBConnect

ion.getConnection().

Im Projektkontext bezeichnet DataSource eine Datenbankverbindung in IntelliJ IDEA (nicht im Java-Code).

Hier ist der genaue Ablauf in IntelliJ:

1. In IntelliJ: Rechtsklick auf das Projekt → Database → Plus-Symbol → Data Source → MySQL.
2. Host (localhost), Port (3306), Datenbank (lagerverwaltung), User und Password werden eingetragen.
3. IntelliJ verwendet diese Konfiguration, um SQL-Statements direkt im Editor auszuführen und Tabellen zu visualisieren.

Im Java-Code selbst verwendet das Projekt kein j

avax.sql.DataSource, sondern DriverManager

ager.getConnection() innerhalb von DBConnect

ion.getConnection().

Aufzählungstyp in Java — eine feste Menge benannter Konstanten.

Hier ist der genaue Ablauf im Projekt:

1. public enum Bewegungstyp { EINGANG, AUSGANG, TRANSFER } — die drei

Enum

java.lang

Error (IntelliJ)

java.lang / IntelliJ

Konstanten entsprechen den MySQL-ENUM-Werten der Spalte bewegungstyp.

2. Beim Schreiben in die Datenbank: `stmt.setString(3, b.getBewegungstyp().name())`
— `.name()` gibt den Enum-Namen als String zurück.

3. Beim Lesen aus der Datenbank:
`BewegungsTyp.valueOf(rs.getString("bewegungstyp"))`
— `.valueOf()` wandelt den String zurück in den Enum-Wert.

Vorteil: Tippfehler bei Bewegungstypen werden zur Kompilierzeit erkannt, nicht erst zur Laufzeit.

Im Projekt steht Error primär für den sichtbaren Fehlerzustand im IntelliJ-Editor (Error Stripe).

Hier ist die Bedeutung:

1. Rote Markierungen im Error Stripe (schmaler Balken rechts im Editor) =

Kompilierungsfehler — das Programm kann nicht gestartet werden.

2. Gelbe Markierungen = Warnungen — das Programm kompiliert, aber es gibt potenzielle Probleme.

3. In Java bezeichnet `java.lang.Error` die Oberklasse schwerwiegender

FXCollections

JavaFX

Laufzeitprobleme wie
OutOfMemoryError — im
normalen Projektbetrieb treten
diese nicht auf.

Klicken auf eine Markierung
im Error Stripe springt direkt
zur betroffenen Zeile.

Factory-Klasse für JavaFX-
Collections — erzeugt Obser-
vableList-Instanzen.

Hier ist der genaue Ablauf im
Projekt:

1. private final Observabl
eList<BestandView>
bestandListe = FXCollections.o
bservableArrayList() —
erzeugt die Liste, die direkt an
tabBestand gebunden ist.

2. Nach jedem
Datenbankzugriff: bestandList
e.setAll(ergebnisse) — ersetzt
alle Einträge und löst
automatisch die Aktualisierung
der TableView aus.

3. In anzeigeFiltern():
FXCollections.o
bservableArrayList() erstellt
eine temporäre gefilterte Liste,
ohne die Datenbank erneut
abzufragen.

FXCollections ist die einzige
Möglichkeit, eine
ObservableList zu erzeugen —
new ObservableList<>()
funktioniert nicht, da es ein
Interface ist.

Group (Maven)

Maven / Projektstruktur

Im Projektkontext bezeichnet

Insets

JavaFX Geometry

Group die Maven-Group-ID de.doit — nicht die JavaFX-Klasse Group.

Hier ist der genaue

Zusammenhang:

1. Beim Anlegen des IntelliJ-Projekts: Group ID = de.doit, Artifact ID = lagerverwaltung.

2. Die Group ID spiegelt sich im Package-Namen aller Klassen wider: de.doit.model, de.doit.dao, de.doit.controller, de.doit.main.

3. In pom.xml: `<groupId>de.doit</groupId>` und `<artifactId>lagerverwaltung</artifactId>`.

Die Kombination Group ID + Artifact ID identifiziert das Maven-Projekt eindeutig.

JavaFX-Geometrie-Klasse für Innenabstände (Padding) von Layout-Containern in main.fxml.

Hier ist der genaue Ablauf:

1. `<?import javafx.geometry.Insets?>` importiert die Klasse am Anfang der FXML-Datei.

2. `<padding><Insets top="10" right="10" bottom="10" left="10"/></padding>` wird innerhalb jeder Tab-VBox gesetzt.

3. Dies verhindert, dass Steuerelemente direkt am Rand des Tab-Containers kleben.

LoadException

JavaFX FXML (Exception)

Insets kann auch asymmetrisch gesetzt werden, z.B. nur oben und unten: `<Insets top="10" bottom="10"/>`.

Exception, die auftritt, wenn FXMLLoader die FXML-Datei nicht verarbeiten kann.

Hier ist der genaue Ablauf:

1. FXMLLoader loader = new FXMLLoader(getClass().getResource("/fxml/main.fxml")) — wenn der Pfad falsch ist, gibt getResource() null zurück.

2. loader.load() wirft dann eine LoadException mit der Meldung "Location is not set".

3. Als caused by-Ursache steht oft eine ClassNotFoundException

assNotFoundException (falscher Klassenname in fx:controller) oder NullPointerException (fehlender führender Slash im Pfad).

Lösung: Pfad mit führendem Slash prüfen

(/fxml/main.fxml),

Klassenname in fx:controller buchstabengenau mit der Klasse vergleichen.

Node

JavaFX Basisklasse

Abstrakte JavaFX-Basisklasse für alle sichtbaren Elemente im Szenegraph.

Hier ist der Zusammenhang im Projekt:

1. Jedes Steuerelement in

NullPointerException

java.lang (Exception)

main.fxml — Button, Label,
TableView, ComboBox,
TextField — ist ein Node.

2. Tab.setContent(Node)

erwartet einen Node als Inhalt
— jede VBox in einem Tab ist
ein Node.

3. Der Szenegraph ist eine
Baumstruktur: Stage enthält
Scene, Scene enthält den
Wurzel-Node (VBox), der
wiederum Kind-Nodes enthält.
Methoden wie setVisible(),
setDisable() und setStyle() sind
in Node definiert und damit für
alle JavaFX-Steuerelemente
verfügbar.

Exception bei Zugriff auf eine
null-Referenz — häufigste
Laufzeitausnahme beim
Projektstart.

Hier ist der genaue Ablauf der
häufigsten Ursache im Projekt:

1. getClass().getResource
("fxml/main.fxml") — ohne
führenden Slash gibt

getResource() null zurück, weil
der Pfad relativ zur Klasse
gesucht wird.

2. new FXMLLoader(null) —
der FXMLLoader bekommt
null als URL.

3. loader.load() wirft dann eine
NullPointerException.

Korrekte Schreibweise: getClass().getResource(
"/fxml/main.fxml") — mit

Object

java.lang

führendem Slash sucht Java vom Classpath-Wurzel aus.

Implizite Basisklasse aller Java-Klassen — auch alle Model-Klassen des Projekts erben von ihr.

Hier ist der Zusammenhang im Projekt:

1. Material.java, Kategorie.java, Lieferant.java etc. sind Objekte im objektorientierten Sinne — Instanzen dieser Klassen repräsentieren je einen Datensatz.

2. Das DAO-Pattern (Data Access Object) bezeichnet Klassen wie MaterialDAO, KategorieDAO etc., die Objekte aus der Datenbank lesen und zurückgeben.

3. toString() ist eine Methode von Object — alle Model-Klassen überschreiben sie mit @Override für lesbare Darstellungen in ComboBoxen. Object selbst wird im Code nicht direkt verwendet — es ist der gemeinsame Nenner aller Klassen.

Parent

JavaFX Basisklasse

Abstrakte JavaFX-Klasse für alle Container, die Kinder-Elemente aufnehmen können. Hier ist der genaue Ablauf im Projekt:

1. Parent root = loader.load() — der FXMLLoader analysiert

PropertyValueFactory

JavaFX TableView

main.fxml und gibt das Wurzelement (die äußerste VBox) als Parent-Referenz zurück.

2. Scene scene = new Scene(root, 900, 600) — die Scene erwartet als erstes Argument ein Parent-Objekt.

3. Parent ist abstract — konkrete Unterklassen VBox, HBox, GridPane, AnchorPane und TabPane werden in FXML verwendet.

Parent selbst kann nicht instanziiert werden — es ist der gemeinsame Typ aller Container.

Verbindet eine TableColumn mit einem Getter-Feld der Datenobjekte via Reflection. Hier ist der genaue Ablauf im Projekt:

1. colMaterial.setCellValueFactory(new PropertyValueFactory<>("materialName")) — der String "materialName" verweist auf den Getter getMaterialName() der Klasse BestandView.

2. JavaFX sucht zur Laufzeit per Reflection nach getMaterialName() — der Name muss exakt mit dem Getter übereinstimmen (Groß-/Kleinschreibung!).

3. Für jede Zeile in tabBestand

Region

JavaFX Layout

wird `getMaterialName()` aufgerufen und der Rückgabewert in der Spalte angezeigt.

Im Projekt werden so alle 6 Spalten (`colMaterial`, `colEinheit`, `colLager`, `colBestand`, `colMindest`, `colStatus`) mit den Gettern von `BestandView` verbunden.

JavaFX-Layout-Element ohne sichtbaren Inhalt — wird als Abstandshalter (`Spacer`) in `HBox` eingesetzt.

Hier ist der genaue Ablauf im Projekt:

1. `<?import javafx.scene.layout.Region?>` importiert die Klasse am Anfang von `main.fxml`.

2. `<Region HBox.hgrow="ALWAYS"/>` wird innerhalb einer `HBox` platziert — `HBox.hgrow="ALWAYS"` lässt die Region allen freien horizontalen Platz einnehmen.

3. Das danach folgende `lblWarnungAnzahl` wird dadurch an den rechten Rand der `HBox` gedrückt.

Ohne die Region würden alle Elemente linksbündig angeordnet — Region ist das Standard-Muster für rechtsbündige Elemente in `HBox`-Layouts.

RowFactory

JavaFX TableView

Ermöglicht die individuelle Gestaltung jeder Zeile einer TableView.

Hier ist der genaue Ablauf im Projekt:

1. tabBest

and.setRowFactory(tv -> new Table Row<BestandView>() { ... }) — das Lambda erzeugt für jede Zeile eine neue TableRow-Instanz.

2. In der überschriebenen updateItem(BestandView item, boolean empty)-Methode wird geprüft: if (item != null && item.isWarnung()).

3. Wenn isWarnung() == true (Bestand < Mindestbestand): setStyle("- fx-background-color: #FADBD8;") — die Zeile wird rot eingefärbt; sonst: setStyle("").

So werden kritische Bestände visuell hervorgehoben, ohne die Daten zu verändern.

Short (Short-Circuit)

java.lang / Operator

Short taucht im Projekt nicht als Datentyp auf, sondern als Begriff der Short-Circuit-Auswertung des ||-Operators.

Hier ist die genaue Erklärung:

1. In DBConnec

tion.getConnection() steht: if (connection == null || connection.isClosed()).

2. Short-Circuit bedeutet:

Wenn connection == null bereits true ist, wird c

SQLIntegrityConstraintViolationException

JDBC (java.sql, Exception)

Statement

JDBC (java.sql)

connection.isClosed() nicht mehr ausgewertet.

3. Das ist entscheidend: Wäre connection tatsächlich null, würde c connection.isClosed() eine NullPointerException werfen.

Durch Short-Circuit wird diese NPE verhindert — der zweite Ausdruck wird nie ausgeführt, wenn der erste bereits true ist.

Exception bei Verletzung einer Fremdschlüssel-Bedingung in der Datenbank.

Hier ist der genaue Ablauf im Projekt:

1.

materialDAO .speichern(material) führt ein INSERT in materialien mit einer kategorie_id aus.

2. Existiert die kategorie_id nicht in der Tabelle kategorien, verweigert MySQL den INSERT.

3. Der JDBC-Treiber wirft eine SQLIntegrityConstraintViolationException: "Cannot add or update a child row: a foreign key constraint fails (lagerverwaltung.material, CONSTRAINT fk__material_kategorie ...)".

Lösung: sicherstellen, dass die referenzierte kategorie_id in der Tabelle kategorien existiert.

Interface aus java.sql — wird

TableRow

JavaFX TableView

im Projekt ausschließlich als
Konstantenquelle verwendet,
nicht für direkte SQL-
Statements.

Hier ist der genaue Ablauf:

1. In allen DAO-Klassen wird
beim INSERT benötigt: conn.p
repareStatement(sql,
Statement.RETUR
N_GENERATED_KEYS).

2. Statement.RET
URN_GENERATED_KEYS
ist eine Konstante (Wert: 1),
die dem JDBC-Treiber mitteilt,
die automatisch generierte ID
zurückzugeben.

3. Nach dem INSERT:
ResultSet keys =
stmt .getGeneratedKeys();
keys.next(); int id =
keys.getInt(1) — so bekommt
das DAO-Objekt die neue
Datenbank-ID.

Direkte SQL-Statements (Sta
tement.execute(sql)) werden
nicht verwendet — das Projekt
setzt ausschließlich

PreparedStatement ein, um
SQL-Injection zu verhindern.

Repräsentiert eine einzelne
Zeile in einer TableView und
wird über die RowFactory
angepasst.

Hier ist der genaue Ablauf im
Projekt:

1. tabBest
and.setRowFactory(tv -> new

TextInputDialog

JavaFX UI

TableRow<BestandView>()
{ ... }) — für jede Zeile erzeugt
JavaFX eine TableRow<BestandView>-Instanz.

2. up dateItem(BestandView
item, boolean empty) wird von
JavaFX für jede Zeile
aufgerufen, wenn sich die
Daten ändern oder die Tabelle
neu gerendert wird.

3. this.setStyle(...) auf der
TableRow setzt das CSS-
Styling für die gesamte Zeile
— im Projekt rot für
Warnungszeilen (#FADBD8),
sonst zurückgesetzt mit "".

TableRow steuert das
Erscheinungsbild der Zeile,
nicht der einzelnen Zellen.

Fertige JavaFX-Dialog-
Variante für einzeilige
Texteingaben in einem Popup-
Fenster.

Hier ist der genaue Ablauf:

1. new TextInputDialog(defaultText) erzeugt
einen Dialog mit einem
vorausgefüllten TextField.

2. showAndWait() blockiert
den JavaFX-Thread, bis der
Benutzer bestätigt oder
abbricht.

3. Optional<String> result = dialog.showAndWait();
result.ifPresent(text -> { ... })
wertet die Eingabe sicher aus.

Im Projekt werden

Timestamp

JDBC (java.sql)

Texteingaben über direkte Text
Field-Steuererelemente in
main.fxml realisiert —
TextInputDialog ist die
Alternative für Eingaben in
separaten Popup-Fenstern.
JDBC-Datentyp für
Datums-/Zeitwerte —
Bindeglied zwischen java.time
(LocalDateTime) und der
Datenbank.
Hier ist der genaue Ablauf in
BestandsbewegungDAO:
1. Beim Schreiben:
stmt.setTimestamp(5,
Timestamp.valueOf(b.getDatum())) —
Timestamp.valueOf(LocalDateTime)
konvertiert den Java-
LocalDateTime-Wert in einen
JDBC-kompatiblen
Timestamp.
2. Beim Lesen: rs.g
etTimestamp("datum
").toLocalDateTime() —
toLocalDateTime() konvertiert
zurück in LocalDateTime.
3. Ist b.getDatum() null:
Timestamp.
valueOf(b.getDatum() != null ?
b.getDatum() :
LocalDateTime.now()) — so
wird immer ein gültiger
Zeitstempel gespeichert.
Timestamp ist notwendig, weil
JDBC kein natives

Window

JavaFX Basisklasse

LocalDateTime versteht — die Konvertierung erfolgt explizit. Abstrakte JavaFX-Oberklasse für alle Fenster-Objekte — Stage ist die verwendete konkrete Unterklasse im Projekt.

Hier ist der Zusammenhang:

1. public void start(Stage primaryStage) — Stage erbt von Window und repräsentiert das Hauptfenster der Lagerverwaltung.
2. primaryStage.setTitle("Lagerverwaltung Krankenhaus") — setTitle() ist in Window definiert und damit von Stage geerbt.
3. primaryStage.show() macht das Fenster sichtbar — show() ist ebenfalls in Window definiert.

Window selbst kann nicht instanziiert werden — im JavaFX-Programm wird immer Stage verwendet.

ZonedDateTime

java.time

Datums-/Zeitklasse mit Zeitzone aus dem java.time-Paket — im Projekt nicht direkt verwendet, aber konzeptuell relevant.

Hier ist der Unterschied zur verwendeten LocalDateTime:

1. LocalDateTime.now() — aktuelle Zeit ohne Zeitzone, verwendet in Be standsbewegung.datum —

ausreichend für ein lokales
Krankenhaussystem.

2. `ZonedDateTime.now()` —
aktuelle Zeit mit Zeitzone,
notwendig für Systeme, die in
mehreren Zeitzonen betrieben
werden.

3. Konvertierung:

`ZonedDateTime.now`
`()`.`toLocalDateTime()` —
entfernt die Zeitzone, wenn nur
`LocalDateTime` benötigt wird.
Das Projekt setzt bewusst
`LocalDateTime` ein, weil ein
Krankenhaus-Lagersystem
keine Zeitzonekomplexität
benötigt.